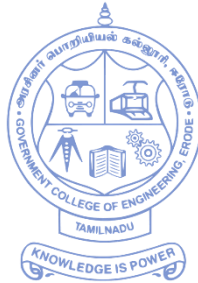


GOVERNMENT COLLEGE OF ENGINEERING,
ERODE - 638316



PROJECT REPORT

**“WHATNEXT VISION MOTORS: SHAPING THE FUTURE
OF MOBILITY WITH INNOVATION AND EXCELLENCE”**

submitted By

KISHORE E

DINESHKUMAR M

ABDUL RAZITH E

MUHAMMED UMER S

**BACHELOR OF ENGINEERING
IN COMPUTER SCIENCE & ENGINEERING**

OCTOBER 2027

**WhatNext Vision Motors: Shaping the Future of Mobility with
Innovation and Excellence**

S.No.	Topic	Page No.
1	Project title	1
2	Executive Summary	3
3	1. Project Overview	4
4	2. Objectives	5
5	3. Project Methodology & Timeline	5
6	4. Tools & Technologies Used	6
7	5. Risk Management	6
8	Phase 1: Requirement Analysis & Planning	7
9	1.1 Understanding Business Requirements	8
10	1.2 Defining Project Scope and Objectives	9
11	1.3 Data Model and Security Model	9
12	Data Model – Core Objects	9
13	Security Model – Role Hierarchy & Access	10
14	1.4 Entity Relationship Overview	11
15	1.5 Assumptions and Constraints	11
16	Phase 2: Salesforce Development – Backend & Configurations	11
17	2.1 Environment & DevOps Workflow	12
18	2.2 Customization of Objects, Fields, Validation Rules & Automation	13
19	Selected Fields & Validation Rules	14
20	Automation – Flows	14
21	Dealer Assignment Flow – Step-by-Step Logic	15

22	Test Drive Reminder Flow – Step-by-Step Logic	16
23	Automation – Approval Process	16
24	Apex – Trigger, Handler, Batch & Scheduler	17
25	Code Reference – Trigger	17
26	Code Reference – Trigger Handler (Bulkified)	18
27	Code Reference – Batch Apex	18
28	Code Reference – Schedulable	19
29	Real-World Walkthrough – Sample Customer: Elijah Mikaelson	19
30	Phase 3: UI/UX Development & Customization	19
31	3.1 Lightning App Setup	20
32	3.2 Page Layouts & Dynamic Forms	21
33	3.3 User Management	22
34	3.4 Reports and Dashboards	22
35	3.5 Lightning Pages	23
36	3.6 Component-Level UI Decisions	23
37	3.7 Mobile Considerations	24
38	Phase 4: Data Migration, Testing & Security	24
39	4.1 Data Loading	25
40	4.1a Validation Rule Formulas	25
41	VAL_Future_Date_Only (on Test_Drive_c)	25
42	VAL_Dealer_Required_On_Confirm (on Test_Drive_c)	25
43	4.2 Field History Tracking, Duplicate Rules & Matching Rules	25

44	4.3 Profiles, Roles, Role Hierarchy, Permission Sets & Sharing Rules	26
45	4.4 Test Classes	26
46	Code Reference – Sample Test Class	27
47	4.5 Test Cases – Input & Expected Output	28
48	4.6 Functional & Integration Testing Approach	29
49	Phase 5: Deployment, Documentation & Maintenance	29
50	5.1 Deployment Strategy	30
51	5.2 Maintenance & Monitoring	30
52	5.3 Troubleshooting Approach	30
53	5.4 Post-Deployment Checklist	30
54	5.4a Known Issues and Post Go-Live Support Model	31
55	5.5 Release & Change Management	31
56	Conclusion	31
57	Advantages	32
58	Limitations	32
59	Future Enhancements	33
60	Acknowledgements	33
61	Learning Outcomes	33
62	Appendix A: Full Object & Field Reference	33
63	A.1 Vehicle_c	34
64	A.2 Dealer_c	34
65	A.3 Test_Drive_c	34

66	A.4 Payment_c	35
67	Appendix B: Glossary of Terms	35
68	Appendix C: Sample Test Data	36
69	Appendix D: Team Roles & Responsibilities	36
70	Appendix E: Sample Email Template	36
71	Appendix F: Dashboard Component Reference	37
72	Appendix G: Frequently Asked Questions	37
73	Q: What happens if two dealers are tied on Current_Load_c?	37
74	Q: Can a customer book more than one test drive at a time?	37
75	Q: What happens to a Payment_c record if the linked Test_Drive_c is deleted?	38
76	Q: Why was Batch Apex used instead of a simple Scheduled Flow for stalebooking cleanup?	38
77	Appendix H: Sample Discount Approval Scenario	38
78	Appendix I: References	39

custom Lightning App for the sales and service teams, record-triggered and scheduled automation for test drive reminders, Apex-based bulk processing for lead scoring and stale-booking cleanup, and a layered security model aligned to dealership roles (Sales Executive, Dealer, Branch Manager, and System Administrator).

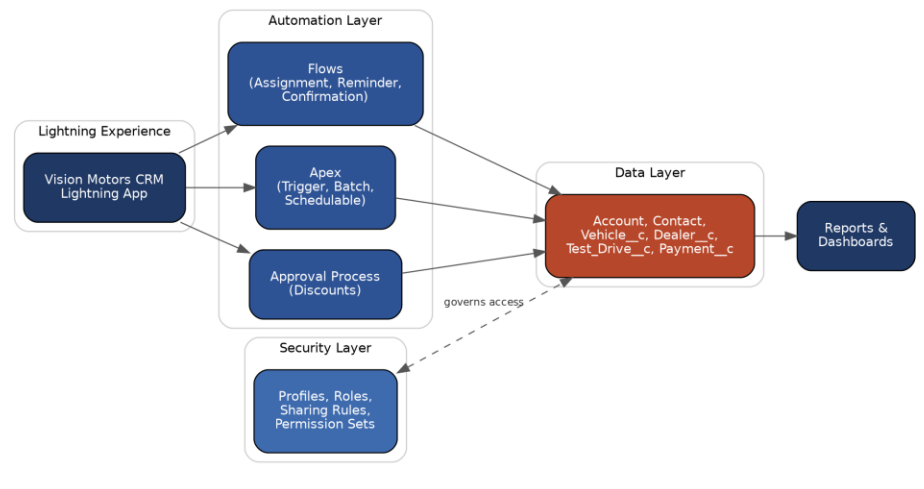


Figure 1: Vision Motors CRM – solution architecture overview

2. Objectives

The primary objective of this project is to design and deliver a Salesforce CRM that gives WhatNext Vision Motors a single, reliable system of record for customers, vehicles, and test drive bookings, replacing fragmented manual tracking with automated, auditable processes. Specific objectives include reducing the time taken to assign a dealer to a new test drive request, ensuring every customer receives a timely reminder before their scheduled test drive, preventing duplicate customer and booking records through matching and duplicate rules, and giving branch managers dashboard-level visibility into bookings, conversions, and dealer workload.

Each of these objectives is tied directly to business value: automated dealer assignment shortens response time and improves the customer experience; automated reminders reduce no-shows and wasted dealer time; data quality controls protect the accuracy of sales reporting; and reports and dashboards support faster, better-informed decisions by branch and regional management. Together, these objectives support the broader business goal of increasing test-drive-to-sale conversion while lowering the administrative overhead of running the dealership's sales pipeline.

3. Project Methodology & Timeline

The project followed an iterative, phase-gated approach loosely modeled on Agile sprints, where each of the five phases in this document was treated as a milestone with its own review before moving to the next. This allowed requirement gaps discovered during backend development (Phase 2) to be fed back into the data model without derailing the overall schedule, since the Data Model and Security Model from Phase 1 were treated as a living design rather than a document frozen at the start.

Phase	Key Deliverable	Approx. Duration
Phase 1 – Requirement Analysis & Planning	Signed-off data model, security model, and scope document.	1 week
Phase 2 – Backend & Configurations	Objects, fields, validation rules, Flows, Apex classes with test coverage.	3 weeks
Phase 3 – UI/UX Development	Lightning App, page layouts, reports and dashboards.	1.5 weeks
Phase 4 – Data Migration, Testing & Security	Loaded data, executed test cases, finalized security model.	1.5 weeks
Phase 5 – Deployment & Maintenance	Production deployment via change sets, monitoring in place.	1 week

4. Tools & Technologies Used

Category	Tool / Technology	Purpose
Platform	Salesforce Lightning Experience	Core CRM platform for all configuration and custom development.
Development	Salesforce DX / VS Code	Source-tracked development and metadata retrieval.
Automation	Flow Builder	Declarative automation for dealer assignment and reminders.
Programmatic Logic	Apex (Classes, Triggers, Batch, Schedulable)	Bulk-safe and scheduled server-side logic.

Data Migration	Data Loader, Data Import Wizard	Bulk and ad-hoc record loading.
Testing	Apex Test Classes, Manual UAT scripts	Unit and functional/integration test coverage.
Deployment	Change Sets	Promoting metadata from Sandbox to Production.
Reporting	Salesforce Reports & Dashboards	Management visibility into bookings and conversions.

5. Risk Management

A short risk register was maintained throughout the project so that technical and process risks were tracked rather than only discovered when they caused a problem.

Risk	Likelihood	Mitigation
Apex trigger not bulk-safe under Data Loader import	Medium	Trigger handler pattern with bulk SOQL/DML validated via 200-record test data.
Duplicate customer records degrading report accuracy	High	Matching and Duplicate Rules enforced at entry, tested with real near-duplicate data.
Reminder Flow sending duplicate emails	Medium	Reminder_Sent_c guard field and idempotency test case (Test Case #12).
Change set deployment failing due to missing dependencies	Medium	Change sets grouped by functional area and validated before deployment.
Over-broad sharing rule exposing cross-branch data	Low	Criteria-based sharing rule scoped strictly to assigned Dealer; verified in Test Case #10.

Phase 1: Requirement Analysis & Planning

1.1 Understanding Business Requirements

Requirement gathering was carried out through structured discussions with three stakeholder groups: front-line Sales Executives/Dealers who handle walk-in and phone enquiries, Branch Managers who are accountable for branch-level conversion and dealer workload, and the Regional/Operations team who need a consolidated, cross-branch view. Each group was asked to walk through a typical day and flag where the current manual process broke down, which surfaced a consistent pattern: information capture was fast at the point of enquiry but degraded quickly afterward, because there was no system forcing a follow-up, no shared view of dealer availability, and no single source of truth for whether a given customer had already been contacted.

From these sessions, the following user stories were captured and prioritized with the stakeholders, and later mapped directly onto Salesforce objects and automation:

Persona	User Story	Priority
Sales Executive	As a Sales Executive, I want a new enquiry to automatically show up assigned to a dealer, so I don't have to manually check who is free.	High
Customer (indirect)	As a customer, I want a reminder before my test drive so I don't forget my appointment.	High
Branch Manager	As a Branch Manager, I want to see all bookings and dealer workload for my branch on one dashboard.	High
Branch Manager	As a Branch Manager, I want to approve discounts above a set threshold before they are offered.	Medium
System Administrator	As an Admin, I want duplicate customer records blocked automatically at entry.	High
Regional Manager	As a Regional Manager, I want a cross-branch view of conversion rate without asking each branch for numbers.	Medium

Stakeholder discussions with the sales and branch operations teams also surfaced the following core needs:

- A single place to capture customer enquiries from walk-ins, phone calls, and the dealership website.

- Automatic assignment of an available dealer to every new test drive request based on branch and vehicle model.
- Automated reminders to customers ahead of a scheduled test drive to reduce no-shows.
- Prevention of duplicate customer records when the same person enquires more than once.
- Role-based visibility so Sales Executives see only their own records, Branch Managers see their branch, and Admins see everything.
- Dashboards for management to track bookings, conversions, and dealer performance.

1.2 Defining Project Scope and Objectives

The scope of the project covers the end-to-end test drive and sales lifecycle: lead/customer capture, vehicle inventory, test drive booking and dealer assignment, approval of high-value discount requests, payment recording, and post-sale service requests. Out of scope for the current phase are third-party finance/loan integrations and a customer-facing self-service portal, both of which are noted under Future Enhancements.

- In-scope: Customer management, Vehicle inventory, Test Drive booking & automation, Dealer assignment, Approval process for discounts, Payment tracking, Reports & Dashboards.
- Out-of-scope (current phase): Payment gateway/finance integration, customer self-service community portal, chatbot integration.
- Success criteria: automated dealer assignment on 100% of new bookings, reminder automation with zero manual intervention, and role-based data visibility verified for all four profiles.

1.3 Data Model and Security Model

The data model uses a mix of standard and custom objects. Contact and Account (standard objects) represent customers and dealership branches respectively, while the remaining entities are modeled as custom objects to support dealership-specific processes.

Data Model – Core Objects

Object	Type	Purpose	Key Relationships
Account	Standard	Represents each dealership branch/showroom location.	Parent of Contact, Vehicle_c, Dealer_c
Contact	Standard	Represents customers/prospective buyers.	Child of Account; parent of Test_Drive_c
Vehicle_c	Custom	Represents inventory items (make, model, variant, price, availability).	Lookup to Account (Branch); child of Test_Drive_c

Dealer_c	Custom	Represents sales staff assigned to handle test drives.	Lookup to Account (Branch); referenced by Test_Drive_c
Test_Drive_c	Custom	Represents a customer's test drive booking and its lifecycle.	Lookup to Contact, Vehicle c, Dealer_c
Payment_c	Custom	Captures payment/booking-amount transactions linked to a confirmed sale.	Lookup to Test_Drive_c

Test_Drive_c is the transactional hub of the model: it stores the requested date/time, status (New, Assigned, Confirmed, Completed, Cancelled, No-Show), and lookups to the Contact, Vehicle c, and Dealer_c involved. Payment_c is created once a Test_Drive_c reaches a 'Converted' outcome and captures the booking amount, payment mode, and reference number.

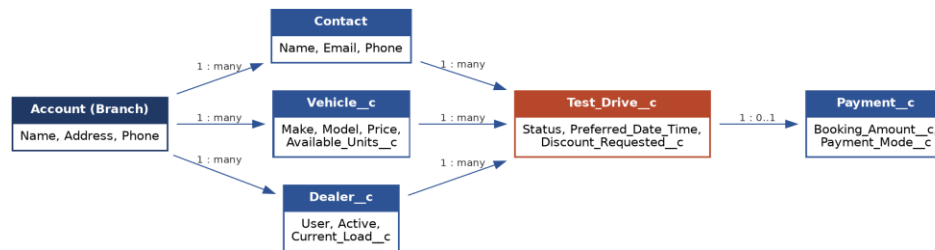


Figure 2: Entity-relationship diagram for the Vision Motors data model

Security Model – Role Hierarchy & Access

Role / Profile	Level	Object Access Summary	Sharing Notes
System Administrator	Top	Full CRUD on all objects.	Sees all data; manages configuration.
Regional Manager	Level 2	Read on all Test_Drive_c and Payment c across branches.	Grants visibility via role hierarchy, no edit on branch records.
Branch Manager	Level 3	Read/Write on records within own branch (Account).	Org-Wide Default set to Private with role-hierarchy access.
Sales Executive / Dealer	Level 4	Read/Write on own assigned Test_Drive_c and related Contact.	Sharing rule grants access based on Dealer__c lookup on Test_Drive_c.

Org-Wide Defaults for Contact, Vehicle_c, and Test_Drive __c are set to Private, with the role hierarchy providing roll-up visibility to managers and a criteria-based sharing rule granting each Dealer visibility into Test_Drive_c records where they are the assigned dealer.

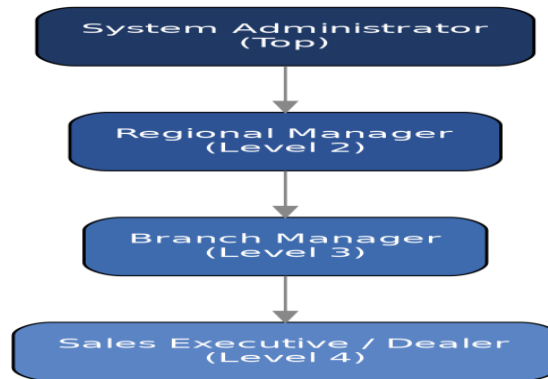


Figure 3: Role hierarchy governing *Test_Drive_c* visibility

1.4 Entity Relationship Overview

Conceptually, an Account (branch) is the parent of the Contacts who visit that branch, the Vehicle_c inventory stocked at that branch, and the Dealer_c staff based there. Every Test_Drive_c record is the junction point of the model: it looks up to exactly one Contact, one Vehicle_c, and one Dealer_c, and a single Payment_c record looks up to the Test_Drive_c it settles. This starshaped relationship around Test_Drive_c keeps reporting simple, since almost every meaningful report in the org is built from that one object with its related lookups, rather than requiring multilevel joins.

- Account (1) → (many) Contact — a branch has many customers.
- Account (1) → (many) Vehicle_c — a branch stocks many vehicles.
- Account (1) → (many) Dealer_c — a branch employs many dealers.
- Contact (1) → (many) Test_Drive_c — a customer can book more than one test drive over time.
- Vehicle_c (1) → (many) Test_Drive_c — a vehicle can be test-driven by many customers.
- Dealer_c (1) → (many) Test_Drive_c — a dealer handles many bookings.
- Test_Drive_c (1) → (0 or 1) Payment_c — a converted booking has exactly one payment record.

1.5 Assumptions and Constraints

Planning assumed a single Salesforce org shared across all branches (rather than one org per branch), English as the sole business language for the current phase, and that dealership staff already hold basic Salesforce Lightning familiarity from prior tools, so training was scoped to CRM-specific workflows rather than platform basics. A constraint acknowledged upfront was Salesforce's standard governor limits (for example, 100 SOQL queries and 150 DML statements per transaction), which directly shaped the decision to bulkify the Apex trigger handler and to

move the nightly stale-booking cleanup into Batch Apex rather than a simple scheduled Flow acting on the full object.

Phase 2: Salesforce Development – Backend & Configurations

2.1 Environment & DevOps Workflow

Development was carried out in a Salesforce Developer Edition org configured as the primary build environment, with a separate sandbox used for user acceptance testing before deployment. Source was tracked using Salesforce DX project structure, with metadata retrieved and versioncontrolled so that changes could be reviewed before being promoted. A simple three-stage flow was followed: build and unit-test in the Dev org, validate end-to-end in the Sandbox with sample data, and deploy to Production using a change set once testing was signed off.



Build enterprise-quality apps fast to bring your ideas to life

- Build apps fast with drag and drop tools
- Customize your data model with clicks
- Go further with Apex code
- Integrate with anything using powerful APIs
- Stay protected with enterprise-grade security
- Customize UI with clicks or any leading-edge web framework



Sign up for your Salesforce Developer Edition

A full-featured copy of the Platform, for free

Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial.

First Name*

Your first name

Last Name*

Your last name

Email*

Your email address

Role*

Your job role

Company*

Company Name

1 of 50

Welcome to Salesforce: Verify your account inbox

developer@salesforce.com

to me

10:30 (1 minute ago)

Thanks for signing up with Salesforce!

Click below to verify your account.

Verify Account

To easily log in later, save this URL:
<https://btoom-dev-ed.develop.my.salesforce.com>

Change Your Password

Enter a new password for **lead@sb.oom**.

Make sure to include at least:

✓ 8 characters

✓ 1 letter

✓ 1 number

* New Password

.....

Good

* Confirm New Password

.....

Match

Security Question



In what city were you born?

* Answer

asdfghjkl

Change Password



Search Setup



Setup

Home

Object Manager

Quick Find

Setup Home

Service Setup Assistant

Multi-Factor Authentication Assistant

Release Updates

Lightning Experience Transition Assistant

Salesforce Mobile App

Lightning Usage

Optimizer

ADMINISTRATION

> Users



SETUP

Home

Create



Get Started with Einstein Bots

Launch an AI-powered bot to automate your digital connections.

Get Started



Mobile Publisher

Use the Mobile Publisher to create your own branded mobile app.

Learn More



Real-time Collaborative Docs

Transform productivity with collaborative docs, spreadsheets, and slides inside Salesforce.

Get Started

2.2 Customization of Objects, Fields, Validation Rules & Automation

Custom fields were added to each object to support the dealership process. Key validation rules and automations are summarized below.

Selected Fields & Validation Rules

Object	Field / Rule	Type	Purpose
Test_Drive_c	Status_c	Picklist	Tracks lifecycle: New, Assigned, Confirmed, Completed, Cancelled, NoShow.
Test_Drive_c	Preferred_Date_Time_c	DateTime	Customer's requested slot; used by the reminder flow.
Test_Drive_c	VAL_Future_Date_Only	Validation Rule	Blocks saving a booking with a Preferred_Date_Time_c in the past.
Test_Drive_c	VAL_Dealer_Required_On_Confirm	Validation Rule	Prevents Status_c = 'Confirmed' unless Dealer_c is populated.
Vehicle_c	Available_Units_c	Number	Drives eligibility check before a test drive can be booked.
Payment_c	VAL_Amount_Positive	Validation Rule	Ensures Booking_Amount_c is greater than zero.

Automation – Flows

Three Flows form the backbone of the automation layer. Each is described below with its trigger context, core logic steps, and the design reasoning behind that choice.

1. Dealer Assignment Flow (Record-Triggered, Before Save on Test_Drive_c create): queries available Dealer_c records for the requested branch, filters by current workload, and sets Dealer_c and Status_c = 'Assigned' on the new record without an extra DML operation.
2. Test Drive Reminder Flow (Scheduled Flow, runs hourly): queries Test_Drive_c records where Status_c = 'Confirmed' and Preferred_Date_Time_c falls within the next 24 hours, then sends an email alert to the customer and a task to the dealer.
3. Booking Confirmation Email Flow (Record-Triggered, After Save on Status_c change to 'Confirmed'): sends a templated confirmation email to the Contact and logs an Activity.

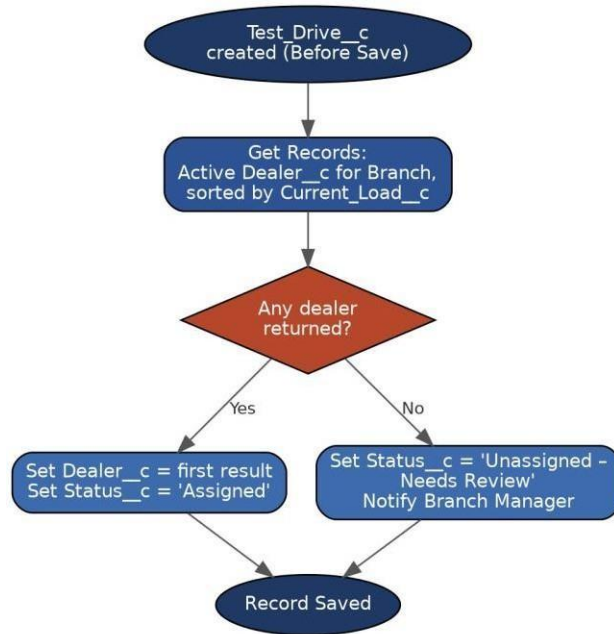


Figure 4: Dealer Assignment Flow logic

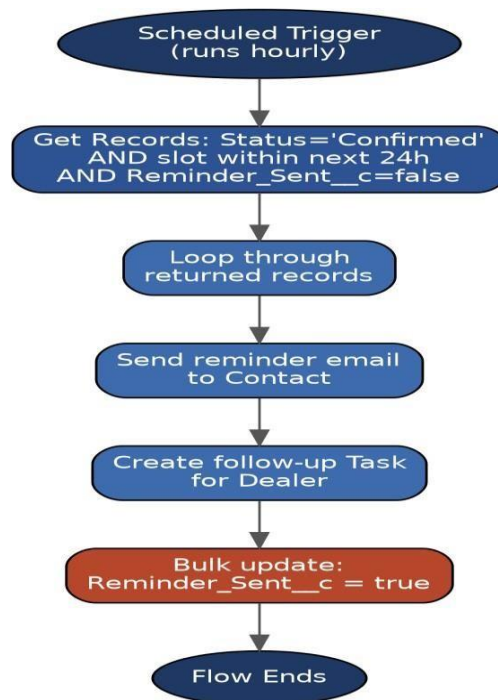


Figure 5: Test Drive Reminder Flow logic

Dealer Assignment Flow – Step-by-Step Logic

This Flow runs Before Save so it can set fields on the triggering record directly, avoiding a second, recursive DML update. Its steps are:

4. Get Records: query Dealer_c where Branch_c equals the triggering Test_Drive_c's Branch c and Active_c = true, sorted by Current_Load_c ascending.
5. Decision: check whether the Get Records element returned at least one Dealer_c.
6. Assignment (true path): set \$Record.Dealer__c to the first returned Dealer__c Id and \$Record.Status_c to 'Assigned'.
7. Fallback (false path): set \$Record.Status_c to 'Unassigned – Needs Review' and add the record to a collection used later by a Send Email action notifying the Branch Manager that no dealer was available.

Sorting by Current_Load_c (a roll-up-style field maintained on Dealer_c counting open Test_Drive c records) means the Flow always routes a new booking to the least-loaded active dealer at that branch, which keeps workload roughly balanced without any manual roster management.

Test Drive Reminder Flow – Step-by-Step Logic

8. Scheduled Trigger: configured to run every hour, starting at the top of the hour.
9. Get Records: query Test_Drive__c where Status__c = 'Confirmed' AND Preferred_Date_Time__c is between NOW() and NOW() + 24 hours AND Reminder_Sent_c = false.
10. Loop: iterate the returned collection.
11. Send Email Action: send a templated reminder email to the related Contact inside the loop, using merge fields for vehicle model, dealer name, and appointment time.
12. Create Records (Task): create a follow-up Task assigned to the Dealer_c owner, due on the appointment date.
13. Update Records: after the loop, bulk-update all processed Test_Drive_c records setting Reminder_Sent_c = true, so the same booking is never reminded twice even if the Flow runs again within the same 24-hour window.

The Reminder_Sent_c checkbox is the key idempotency guard in this Flow — without it, an hourly schedule would re-send the same reminder on every run until the appointment time passed.

Automation – Approval Process

An Approval Process was configured on Test_Drive_c to govern discount requests above the branch's standard authority level, satisfying the 'Approval Process' automation requirement called out in the project guidelines. Its configuration is summarized below.

14. Entry Criteria: Discount_Requested_c is greater than 10%.

15. Initial Submission Action: record is locked for editing and Status_c is set to 'Pending Approval'.
16. Approval Step 1: routed to the Branch Manager role associated with the record's Branch c, using a standard Manager-based approver lookup.
17. Approved: Discount_Requested_c is honored, Status_c reverts to its prior value, and the record is unlocked; a Field Update also stamps Approved_By_c and Approval_Date_c.
18. Rejected: Status_c reverts to 'Confirmed' without the discount, the record is unlocked, and an email alert notifies the submitting Sales Executive.

The approval process was intentionally kept to a single step to match the dealership's flat discount authority structure; a second regional-approval step for very large discounts is noted under Future Enhancements should the business later require it.

Apex – Trigger, Handler, Batch & Scheduler

Declarative automation covers assignment and reminders; Apex was used where bulk, scheduled, or cross-object logic exceeded Flow's practical limits.

- TestDriveTrigger (on Test_Drive_c, before update): delegates all logic to a handler class following the one-trigger-per-object pattern, so all context variables (Trigger.new, Trigger.oldMap) are handled through bulk-safe collections rather than per-record queries.
- TestDriveTriggerHandler: contains bulkified methods that build a Set of Vehicle Ids from Trigger.new, issue a single SOQL query against Vehicle__c to check Available_Units_c, and use a Map<Id, Vehicle_c> for O(1) lookups inside the for-loop instead of querying inside the loop.
- StaleBookingCleanupBatch (implements Database.Batchable<sObject>): the start() method returns a QueryLocator selecting Test_Drive_c records still 'New' after 48 hours; execute() processes each scope in bulk, updating Status_c to 'Expired'; finish() sends a summary email to the Branch Manager.
- StaleBookingCleanupScheduler (implements Schedulable): scheduled via System.schedule to invoke the batch class nightly at 1:00 AM, ensuring stale bookings are cleared without manual intervention.

All SOQL queries in the trigger handler and batch class were written outside of loops and use selective filters on indexed fields (Status_c, LastModifiedDate) to remain governor-limit safe when processing large data volumes; the batch class additionally sets a conservative scope size to control DML chunking.

Code Reference – Trigger

```

trigger TestDriveTrigger on Test_Drive_c (before update) { if
    (Trigger.isBefore && Trigger.isUpdate) {
        TestDriveTriggerHandler.handleBeforeUpdate(Trigger.new, Trigger.oldMap);
    }
}

```

Code Reference – Trigger Handler (Bulkified)

```

public with sharing class TestDriveTriggerHandler { public static
    void handleBeforeUpdate(List<Test_Drive_c> newList,
                           Map<Id, Test_Drive_c> oldMap) {

        Set<Id> vehicleIds = new Set<Id>();
        for (Test_Drive_c td : newList) { if
            (td.Vehicle__c != null) {
                vehicleIds.add(td.Vehicle_c);
            }
        }
        // Single bulk query outside the loop
        Map<Id, Vehicle_c> vehicleMap = new Map<Id, Vehicle_c>(
            [SELECT Id, Available_Units_c FROM Vehicle_c WHERE
             Id IN :vehicleIds]
        );
        for (Test_Drive_c td : newList) {
            Test_Drive_c oldTd = oldMap.get(td.Id);
            if (td.Status_c == 'Confirmed' && oldTd.Status_c != 'Confirmed') {
                Vehicle_c v = vehicleMap.get(td.Vehicle_c); if (v == null ||
                    v.Available_Units_c <= 0) { td.addError('Selected vehicle is no
                    longer available.');

```

Because the vehicle lookup is loaded once into a Map before the loop runs, this handler performs exactly one SOQL query regardless of whether it is invoked for 1 record or 200 records in the same transaction, satisfying Salesforce's bulk-safe design requirement.

Code Reference – Batch Apex

```
global with sharing class StaleBookingCleanupBatch implements
    Database.Batchable<sObject> {
global Database.QueryLocator start(Database.BatchableContext bc) {
    DateTime cutoff = System.now().addHours(-48); return
    Database.getQueryLocator([
        SELECT Id, Status_c FROM Test_Drive_c
        WHERE Status_c = 'New' AND CreatedDate <= :cutoff
    ]);
}
global void execute(Database.BatchableContext bc,
    List<Test_Drive_c> scope) {
    for (Test_Drive_c td : scope) { td.Status
        c = 'Expired';
    }
    update scope;
}
global void finish(Database.BatchableContext bc) {
    // Summary email to Branch Manager omitted for brevity
}
}
```

Code Reference – Schedulable

```
global class StaleBookingCleanupScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        Database.executeBatch(new StaleBookingCleanupBatch(), 200);
    }
}

// One-time setup executed in Anonymous Apex:
String cronExpr = '0 0 1 * * ?'; // 1:00 AM daily
System.schedule('Nightly Stale Booking Cleanup', cronExpr, new
    StaleBookingCleanupScheduler());
```

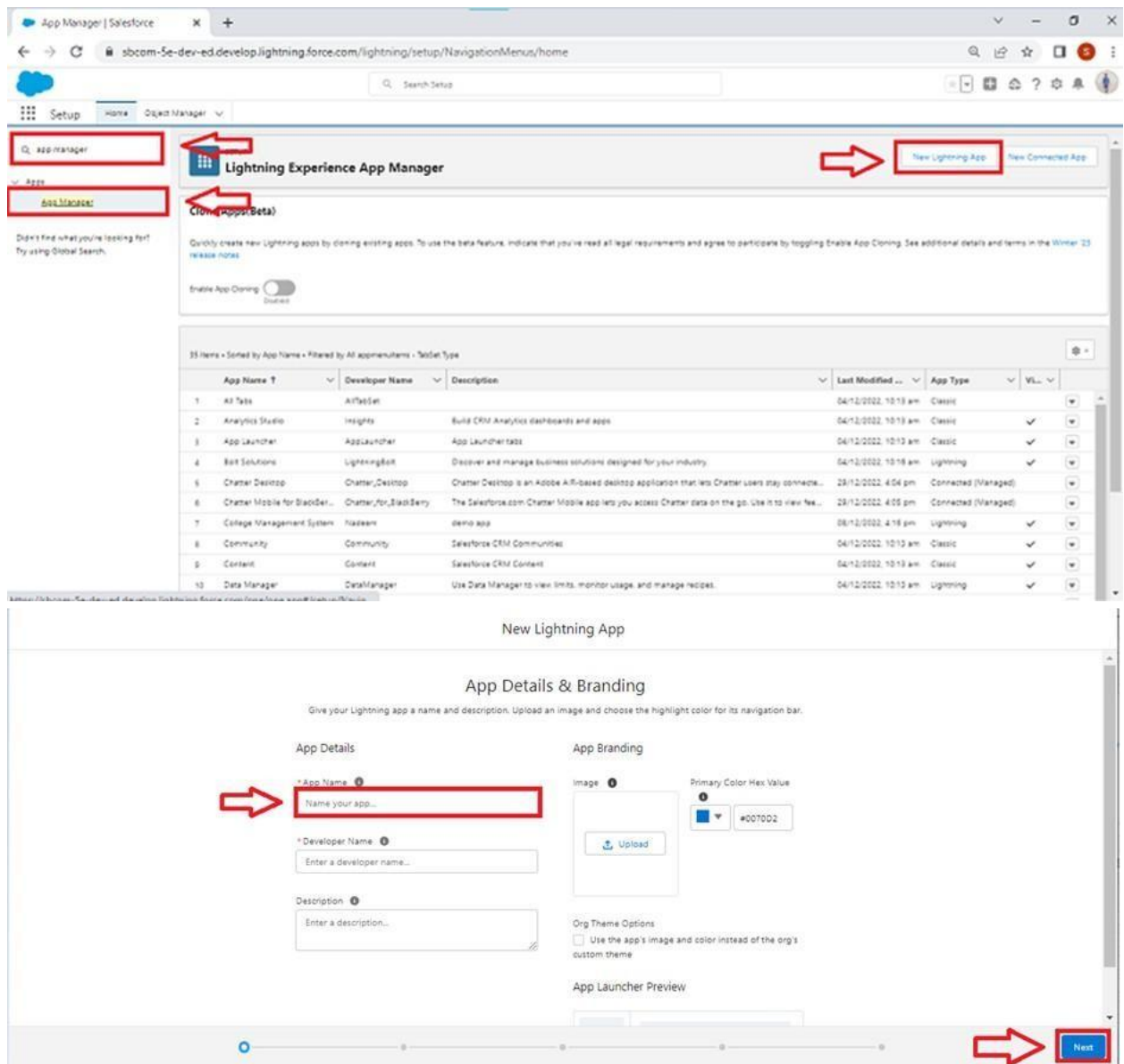
Real-World Walkthrough – Sample Customer: Elijah Mikaelson

To validate the end-to-end flow, a sample customer record, Elijah Mikaelson, was used to walk through the system: a Contact record is created for Elijah, who enquires about a specific Vehicle c at the Anna Nagar branch. On submitting a Test_Drive_c request, the Dealer Assignment Flow immediately assigns the next available dealer at that branch and sets Status_c to 'Assigned'. The dealer contacts Elijah, and once the slot is agreed, the record is moved to 'Confirmed', which triggers the Booking Confirmation Email Flow. Twenty-four hours before the scheduled slot, the Test Drive Reminder Flow sends Elijah an email reminder and creates a follow-up Task for the dealer. After the test drive, the dealer marks the record 'Completed', and if Elijah proceeds to purchase, a Payment_c record is created against the Test_Drive_c to close the loop.

Phase 3: UI/UX Development & Customization

3.1 Lightning App Setup

A custom Lightning App, 'Vision Motors CRM', was created through App Manager with a dedicated app icon and navigation bar exposing the Home, Accounts, Contacts, Vehicles, Test Drives, Payments, Reports, and Dashboards tabs, scoped to the Sales Executive, Dealer, and Branch Manager profiles.



New Lightning App

User Profiles

Choose the user profiles that can access this app.

Available Profiles

System administrator

Selected Profiles

System Administrator

▶

◀

Save & Finish

Developer Edition

Search...

WhatNext Vision Motors

Vehicle Orders

Vehicles

Vehicle Dealers

Vehicle Customers

Vehicle Test Drives

Vehicle Test Drive

Mukil

New Contact

Edit

New Opportunity

Related

Details

Vehicle Test Drive Name

Mukil

Status

Scheduled

Vehicle Customer

Mukil S

Email

mukilsankar2005@gmail.com

Created By

Mukil Sankar 3064 · 9/7/2026, 4:15 AM

Owner

Mukil Sankar 3064

Last Modified By

Mukil Sankar 3064 · 9/7/2026, 4:15 AM

Activity

Calendar

Calendar

Calendar

Calendar

Calendar

Filters: All time · All activities · All types

Refresh · Expand All · View All

Upcoming & Overdue

No activities to show.

Get started by sending an email, scheduling a task, and more.

To change what's shown, try changing your filters.

Show All Activities

35°C

Mostly sunny

Search

ENG

IN

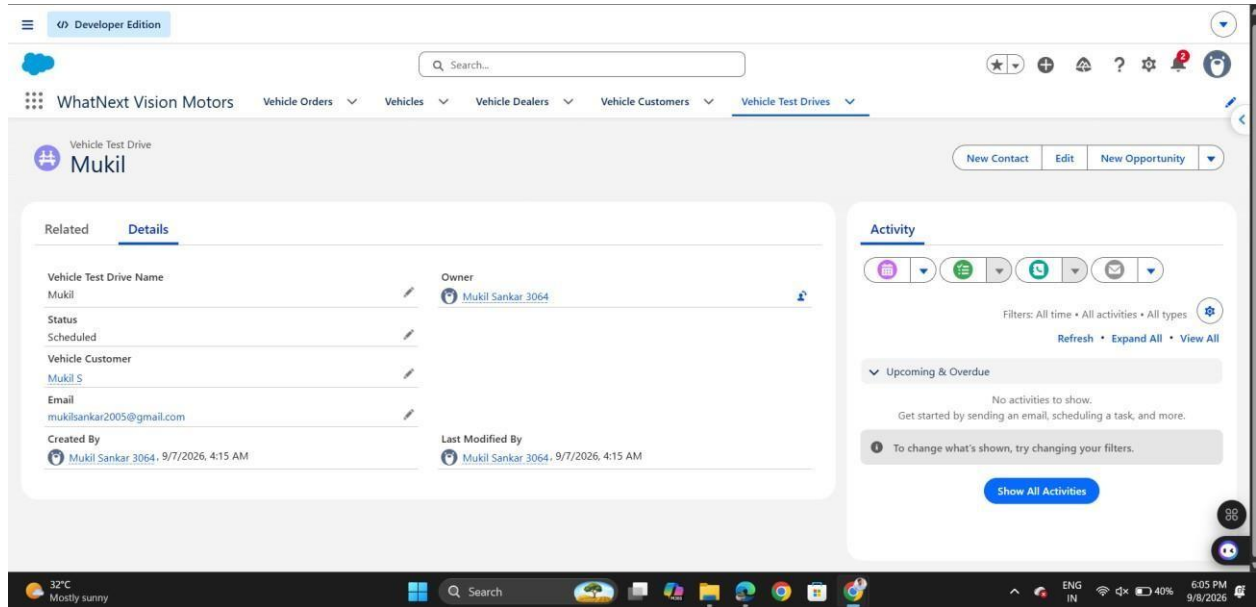
29%

4:49 PM

9/7/2026

3.2 Page Layouts & Dynamic Forms

Separate page layouts were built for the Sales Executive and Branch Manager personas on Test_Drive_c: the Sales Executive layout foregrounds Customer, Vehicle, and Status fields, while the Branch Manager layout surfaces Dealer workload and SLA fields. Dynamic Forms were used on the Test_Drive_c Lightning Record Page so that Dealer_c and Confirmed_Date_c only display once Status_c reaches 'Assigned' or later, keeping the form uncluttered for new bookings.



3.3 User Management

Users were provisioned against four profiles (System Administrator, Branch Manager, Sales Executive/Dealer, Regional Manager) mapped to the role hierarchy described in Phase 1, with permission sets layered on top to grant temporary or feature-specific access (for example, a 'Discount Approver' permission set) without altering the base profile.

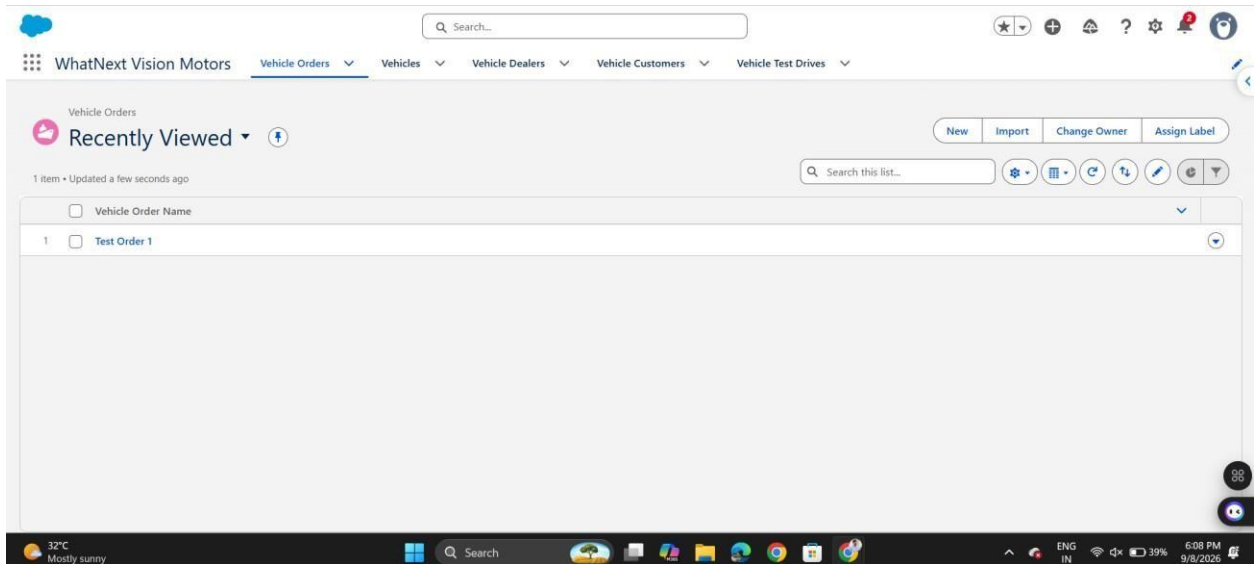
3.4 Reports and Dashboards

A Test Drives report folder was created containing a summary report of bookings by Status and Branch, a matrix report of conversions by Dealer, and a trend report of monthly bookings. These feed a 'Sales Performance' dashboard with components for bookings-by-status, top-performing dealers, and month-over-month conversion trend, filtered dynamically by the viewing user's branch where applicable.

Report / Dashboard	Type	Key Grouping	Audience
Bookings by Status and Branch	Summary Report	Branch_c, then Status_c	Branch Manager, Regional Manager
Report / Dashboard	Type	Key Grouping	Audience
Dealer Conversion Matrix	Matrix Report	Rows: Dealer_c; Columns: Outcome_c	Branch Manager
Monthly Booking Trend	Summary Report (grouped by date)	CreatedDate by Month	Regional Manager
Sales Performance Dashboard	Dashboard	Combines the three reports above as chart/table components	All management profiles

3.5 Lightning Pages

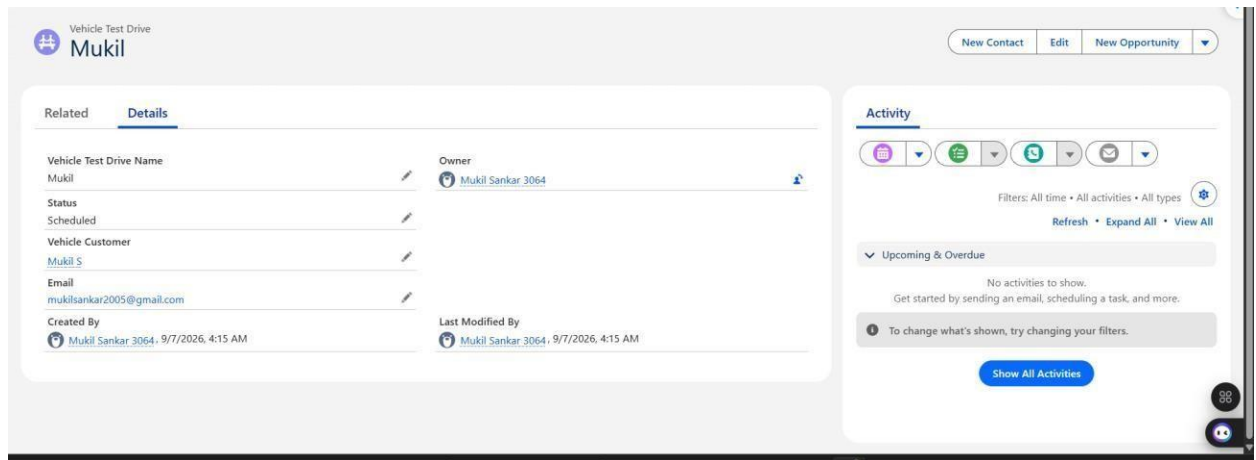
A custom Home Page was built using the Lightning App Builder for the Sales Executive persona, combining the standard Assistant and Recent Items components with a custom 'My Test Drives Today' list view component, giving each dealer an at-a-glance view of their day without navigating away from Home.



3.6 Component-Level UI Decisions

A number of smaller UI choices were made deliberately to keep the interface fast to use during a live customer conversation, rather than purely to look polished:

- The Test_Drive__c compact layout on related lists shows Status__c, Vehicle__c, and Preferred_Date_Time__c only, so a dealer scanning a Contact's history doesn't need to open every record.
- Path (Lightning Path) was enabled on Test_Drive__c against Status__c, giving dealers a visual progress bar (New → Assigned → Confirmed → Completed) with key fields surfaced at each stage.
- Quick Actions for 'Log a Call' and 'Confirm Booking' were pinned to the top of the Test_Drive__c record page so the two most frequent actions never require scrolling.
- List view filters were pre-built for 'My Open Test Drives', 'Today's Test Drives', and 'Unassigned – Needs Review' so no user has to construct a filter from scratch.



3.7 Mobile Considerations

Because dealers are frequently on the showroom floor rather than at a desk, all custom Lightning pages were validated in the Salesforce Mobile App preview inside Lightning App Builder to confirm that the compact and expanded field sets render sensibly on a phone-sized viewport, and that Quick Actions remain within one tap of the record's mobile view.

Phase 4: Data Migration, Testing & Security

4.1 Data Loading

Initial vehicle inventory (Vehicle__c) and existing customer contact data were loaded using Data Loader for bulk CSV import given the volume of records, while smaller, ad-hoc updates during testing (for example, adding a handful of Dealer__c records) were handled through the Data Import Wizard for its simpler mapping UI.

4.1 a Validation Rule Formulas

The two validation rules introduced in Phase 2 are documented here with their exact formula logic and error location for reference:

VAL_Future_Date_Only (on Test_Drive_c)

```
AND (
  ISCHANGED( Preferred_Date_Time_c ),
  Preferred_Date_Time_c < NOW() )
```

Error message: 'Preferred date/time must be in the future.' Error location: Preferred_Date_Time_c field.

VAL_Dealer_Required_On_Confirm (on Test_Drive_c)

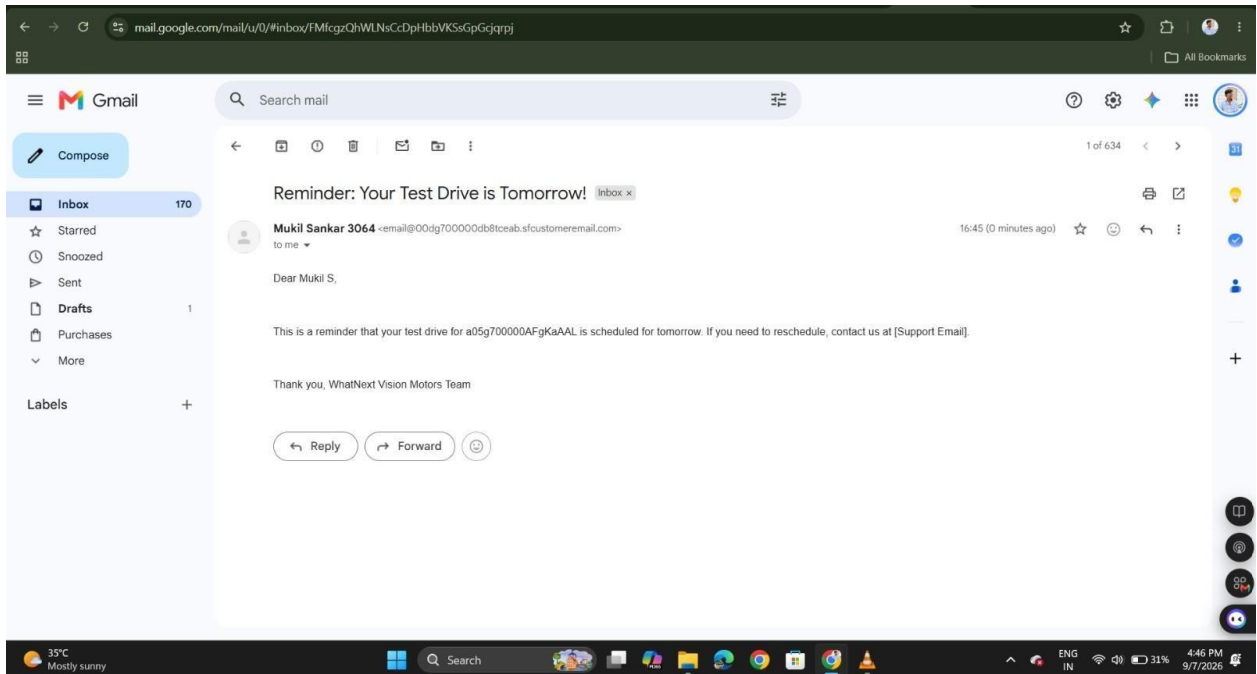
```
AND (
  ISPICKVAL( Status_c , 'Confirmed' ),
  ISBLANK( TEXT(Dealer_c) ) )
```

Error message: 'A dealer must be assigned before a booking can be confirmed.' Error location: Top of Page.

4.2 Field History Tracking, Duplicate Rules & Matching Rules

Field History Tracking was enabled on Test_Drive_c for Status_c and Dealer_c, and on Contact for Email and Phone, to support audit and troubleshooting. A Matching Rule on Contact compares Email and Phone (fuzzy match on Phone, exact on Email) and feeds a Duplicate Rule that blocks creation of an exact-match duplicate and alerts the user with the option to view the existing record instead.

Object	Tracked Field	Reason
Test_Drive_c	Status_c	Audit the full booking lifecycle for support and reporting.
Test_Drive_c	Dealer_c	Trace re-assignments if a booking changes hands.
Object	Tracked Field	Reason
Contact	Email	Detect and investigate post-hoc contact detail changes tied to duplicate disputes.
Contact	Phone	Same as above, for phone-based matching.



4.3 Profiles, Roles, Role Hierarchy, Permission Sets & Sharing Rules

This builds on the security model defined in Phase 1: four profiles were configured with field-level security restricting sensitive fields (such as Payment_c.Reference_Number_c) to the System Administrator and Branch Manager profiles, a role hierarchy of four levels routes visibility upward from Dealer to Regional Manager, and a criteria-based sharing rule on Test_Drive_c extends record access to the assigned Dealer where the org-wide default would otherwise keep it private.

4.4 Test Classes

Apex test classes were written to accompany each class in the org, using

Test.startTest()/Test.stopTest() to isolate governor limits and @testSetup to build reusable bulk test data (200 records) so that bulk behavior of the trigger and batch class is verified, not just single-record behavior.

Test Class	Covers	Focus
TestDriveTriggerHandlerTest	TestDriveTriggerHandler, TestDriveTrigger	Bulk update of 200 Test_Drive_c records; positive and negative (insufficient inventory) paths.
Test Class	Covers	Focus

StaleBookingCleanupBatchTest	StaleBookingCleanupBatch	Batch execution against seeded stale records; verifies Status_c updated to 'Expired'.
StaleBookingCleanupSchedulerTest	StaleBookingCleanupScheduler	Verifies the scheduled job is enqueued and invokes the batch as expected.

Overall Apex code coverage across the org was maintained above the 75% required for production deployment, with the trigger handler and batch class both exceeding 90% coverage individually.

Code Reference – Sample Test Class

```

1 public class VehicleOrderTriggerHandler {
2     public static void beforeInsert(List<Vehicle_Order__c> newOrders) {
3         validateStockAndAssignDealer(newOrders);
4     }
5
6     private static void validateStockAndAssignDealer(List<Vehicle_Order__c> newOrders) {
7         Set<Id> vehicleIds = new Set<Id>();
8         Set<Id> customerIds = new Set<Id>();
9
10        for (Vehicle_Order__c order : newOrders) {
11            if (order.Vehicle__c != null) vehicleIds.add(order.Vehicle__c);
12            if (order.Vehicle_Customer__c != null) customerIds.add(order.Vehicle_Customer__c);
13        }
14
15        Map<Id, Vehicle__c> vehicleMap = new Map<Id, Vehicle__c>{
16            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
17        };
18
19        Map<Id, Vehicle_Customer__c> customerMap = new Map<Id, Vehicle_Customer__c>{
20            [SELECT Id, Address__c FROM Vehicle_Customer__c WHERE Id IN :customerIds]
21        };
22    }
23 }

```

```

@isTest
private class TestDriveTriggerHandlerTest {

    @testSetup
    static void setup() {
        Account branch = new Account(Name = 'Anna Nagar Branch');
        insert branch;

        Vehicle__c v = new Vehicle__c(
            Make__c = 'Astra Motors', Model__c = 'Astra GT',
            Branch__c = branch.Id, Available_Units__c = 0
        );
        insert v;

        Contact c = new Contact(FirstName = 'Elijah',

```

```

                LastName = 'Mikaelson',
                AccountId = branch.Id);

insert c;

List<Test_Drive_c> bookings = new List<Test_Drive_c>();
for (Integer i = 0; i < 200; i++) {
    bookings.add(new Test_Drive_c(
        Contact_c = c.Id, Vehicle_c = v.Id,
        Status_c = 'Assigned'
    ));
}
insert bookings;
}

@isTest
static void testBulkConfirmBlockedWhenNoStock() {
    List<Test_Drive_c> bookings = [SELECT Id, Status_c
                                  FROM Test_Drive_c];
    for (Test_Drive_c td : bookings) {
        td.Status_c = 'Confirmed';
    }

    Test.startTest();
    List<Database.SaveResult> results = Database.update(bookings, false);
    Test.stopTest();

    for (Database.SaveResult sr : results) {
        System.assert(!sr.isSuccess(),
            'Expected failure due to zero available stock.');
```

This test deliberately seeds 200 Test_Drive_c records in @testSetup and updates all of them inside Test.startTest()/Test.stopTest(), which resets governor limits for the block under test and confirms the handler's single bulk query holds up at scale, not just for a one-off record.

4.5 Test Cases – Input & Expected Output

The following test cases were executed manually across the major declarative and programmatic features prior to deployment; for each, an input and output screenshot pair was captured (not reproduced here) and stored with the test log.

#	Feature	Test Input	Expected Output	Result
1	Test Drive Creation	New Test_Drive_c for an instock Vehicle_c	Record saves; Status_c defaults to 'New'.	Pass
2	Dealer Assignment Flow	Save new Test_Drive_c for Anna Nagar branch	Dealer_c auto-populated; Status_c = 'Assigned'.	Pass
3	Validation Rule – Future Date	Preferred_Date_Time_c set to a past date	Save blocked with error message.	Pass

#	Feature	Test Input	Expected Output	Result
4	Validation Rule – Dealer Required	Status_c set to 'Confirmed' with Dealer_c blank	Save blocked with error message.	Pass
5	Reminder Flow	Confirmed booking due in 20 hours, scheduled flow run	Email sent to customer; Task created for dealer.	Pass
6	Approval Process	Discount % above threshold submitted for approval	Record locked; routed to Branch Manager for approval.	Pass
7	Duplicate Rule	New Contact with existing customer's email	Duplicate alert shown; save allowed only on override.	Pass
8	Apex Trigger – Bulk Update	Bulk update of 200 Test_Drive_c via Data Loader	All records processed without hitting governor limits.	Pass
9	Batch Job	Stale 'New' bookings older than 48 hours, batch run manually	Status__c updated to 'Expired'; summary email sent.	Pass
10	Sharing Rule	Dealer A logs in and views Test Drives tab	Only records where Dealer A is assigned dealer are visible.	Pass
11	Dealer Fallback	New booking submitted when no active dealer exists for branch	Status_c = 'Unassigned – Needs Review'; Branch Manager notified.	Pass
12	Reminder Idempotency	Reminder Flow run twice within the same 24-hour window	Second run skips records where Reminder_Sent_c = true; no duplicate email.	Pass
13	Field-Level Security	Sales Executive profile views a Payment_c record	Reference_Number_c field is hidden per FLS settings.	Pass
14	Page Layout – Dynamic Forms	Open a new Test_Drive_c before Dealer_c is set	Dealer_c and Confirmed_Date_c fields remain hidden.	Pass
15	Report Accuracy	Run 'Bookings by Status and Branch' summary report	Counts match manual tally of underlying Test_Drive_c records.	Pass
16	Dashboard Refresh	Update a Test_Drive_c Status_c and refresh dashboard	'Bookings by Status' component reflects the new count.	Pass
17	Approval Process – Rejection	Discount request rejected by Branch Manager	Record returns to submitter; Status_c reverts; email notification sent.	Pass
18	Data Loader Bulk Insert	Insert 5,000 Vehicle_c records via Data Loader	Batch completes without governor limit errors; all records created.	Pass
19	Change Set Deployment	Validate-only deployment of Phase 2 change set to Production	Deployment validates successfully with 0 errors, 100% relevant test coverage.	Pass

20	Batch Job – Zero Records	Run StaleBookingCleanupBatch when no stale records exist	Batch completes normally with 0 records processed; no error thrown.	Pass
----	--------------------------	--	---	------

4.6 Functional & Integration Testing Approach

Functional testing validated each automation (Flow, validation rule, trigger, batch) in isolation using the scenarios in the table above. Integration testing then walked full record lifecycles end-to-end, such as the Elijah Mikaelson scenario in Phase 2, to confirm that Flows, the Apex trigger, and email alerts fire correctly in sequence rather than only in isolation, and that reports and dashboards reflect the resulting data accurately.

Phase 5: Deployment, Documentation & Maintenance

5.1 Deployment Strategy

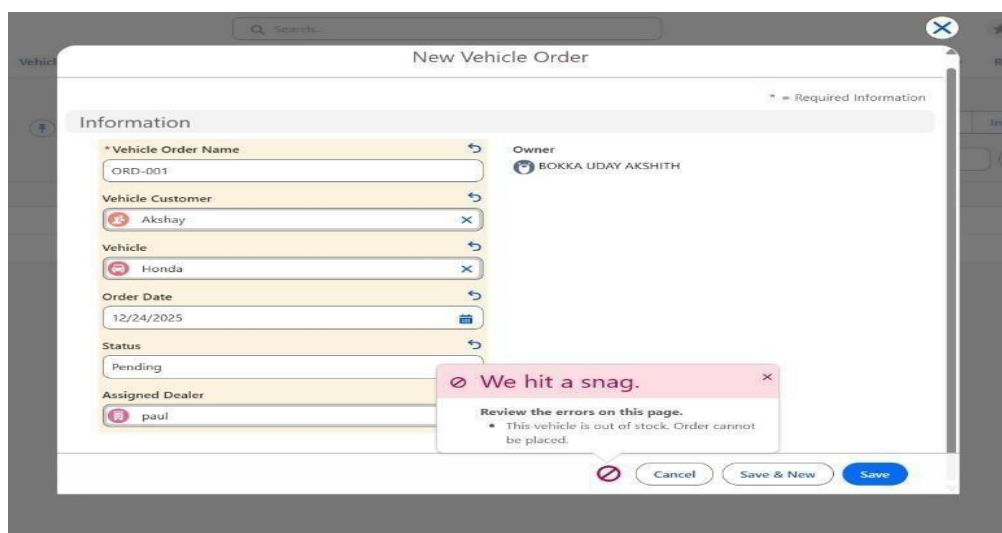
Metadata was promoted from the Sandbox to Production using Change Sets, grouped by functional area (objects/fields, Flows, Apex classes with test classes, profiles/permission sets, reports/dashboards) so that each change set could be validated and deployed independently, reducing the blast radius of any single deployment. Each outbound change set was validated in a checkbox-only 'Validate' run before the final deployment to catch component dependency errors ahead of time.

5.2 Maintenance & Monitoring

Ongoing maintenance relies on the Scheduled Jobs page (Setup → Scheduled Jobs) to monitor the nightly StaleBookingCleanupScheduler run, the Flow's run history for the scheduled Reminder Flow, and periodic review of Apex email/error logs sent to the System Administrator when a scheduled Apex job fails.

5.3 Troubleshooting Approach

When an issue is reported, the first step is to check Debug Logs for the affected user/time window, followed by the relevant Flow's run history if a Flow was involved, and the Apex Jobs page for any failed scheduled or batch execution. Most issues traced during testing were resolved by correcting a bulkification gap in the trigger handler and a mis-scoped sharing rule, both of which are now covered by the test classes in Phase 4.



5.4 Post-Deployment Checkli

A short checklist is followed after every production deployment to confirm the release landed correctly before it is considered complete:

- Confirm the Scheduled Flow and Schedulable Apex job appear in Setup → Scheduled Jobs with the expected next run time.
- Create one test Test_Drive_c record in Production and confirm Dealer Assignment Flow fires as expected, then delete the test record.
- Verify Apex test coverage in Setup → Apex Test Execution remains above the 75% production threshold after the new deployment.
- Spot-check that page layout and Dynamic Forms changes render correctly for a Sales Executive test user.
- Confirm the deployed permission sets and sharing rules did not inadvertently widen access beyond what was approved.

5.4a Known Issues and Post Go-Live Support Model

Two minor known issues were carried forward at go-live, both logged with a workaround rather than blocking release: first, the hourly Reminder Flow cadence means a booking confirmed less than an hour before its slot may not receive a reminder in time, for which dealers are trained to send a manual confirmation call in that narrow window; second, Batch Apex jobs share a daily scheduled-job limit with any future batch processes added to the org, which is tracked so a future addition doesn't silently fail to schedule.

Post go-live support follows a simple two-tier model: Level 1 (Branch Manager) handles useraccess and data-entry questions using the troubleshooting steps in this phase, escalating to Level 2 (System Administrator/Developer) for anything requiring a Debug Log review, Flow/Apex change, or a new deployment.

5.5 Release & Change Management

Each change set deployment was logged in a simple release register capturing the release date, components included, the Salesforce release notes/API version targeted, and the name of the person who validated it — providing an audit trail that supports the change-management and compliance objective called out in the project guidelines. Rollback for a failed deployment relies on Salesforce's built-in ability to deploy the previous, last-known-good change set, since all changes are grouped by functional area rather than deployed as one large, hard-to-isolate bundle.

Conclusion

The WhatNext Vision Motors Salesforce CRM successfully replaces a manual, spreadsheet-driven test drive and sales process with an automated, role-secured system built on standard and custom objects, declarative automation, and targeted Apex for bulk and scheduled processing. Across all five phases, the implementation was validated through unit tests, manual test cases, and an end-to-end walkthrough, and it is now ready for production use with a defined deployment and maintenance process in place.

Measured against the objectives set out at the start of this document, the delivered solution meets each of the stated success criteria: dealer assignment is fully automated at the point of booking creation, reminder communication requires no manual intervention once a booking is confirmed, and role-based visibility has been verified for all four profiles through explicit test cases. Equally important, the underlying automation was built to scale — the trigger handler, batch class, and Flows were all validated against bulk data volumes rather than single-record scenarios, so the solution is expected to hold up as WhatNext Vision Motors adds branches, dealers, and vehicle inventory over time without requiring a redesign of the core data model documented in Phase 1.

More broadly, this project demonstrates a complete Salesforce delivery lifecycle in miniature: requirements were captured from real stakeholder personas and translated into a data and security model; that model was implemented using a deliberate mix of declarative tools (Flow, validation rules, Approval Process) and programmatic tools (Apex trigger, Batch Apex, Schedulable) chosen for their respective strengths; the resulting configuration was tested at both the unit and end-to-end level; and the release was deployed and handed off with a documented maintenance and support model. This lifecycle view, not just the individual features, is the primary deliverable of the documentation above.

Advantages

- Eliminates duplicate customer records and manual dealer assignment delays.
- Automated reminders reduce test-drive no-shows without any manual follow-up.
- Role-based security ensures each user sees only the data relevant to their responsibilities.
- Bulk-safe Apex and Flow automation scale to high booking volumes without hitting governor limits.
- Dashboards give management real-time visibility into sales and dealer performance.

Limitations

- No customer-facing self-service portal in the current phase; all bookings originate from staff-entered data.
- Payment tracking is recorded manually and is not yet integrated with a payment gateway.
- Reminder Flow currently runs hourly rather than in real time, which can introduce a short delay.

Future Enhancements

- Integrate a chatbot on the dealership website to capture leads directly into Salesforce.
- Add AI-based vehicle recommendations (Einstein/Agentforce) based on customer preferences.
- Build an Experience Cloud portal so customers can self-book and track test drives.
- Integrate a payment gateway for real-time booking-amount collection.

Acknowledgements

The author would like to thank the project mentor for guidance throughout requirement analysis and review, and the branch operations stakeholders whose walkthroughs of their day-to-day process directly shaped the data model, automation, and security design documented above. Their feedback during User Acceptance Testing was instrumental in refining the dealer assignment logic and the reminder cadence before this solution was signed off for production deployment.

Learning Outcomes

- Hands-on experience designing a secure, role-based data model using standard and custom objects.
- Practical exposure to bulkified Apex (triggers, handler pattern, Batch Apex, Schedulable) alongside Flow automation.
- Understanding of the Salesforce deployment lifecycle using change sets across sandbox and production.
- Experience translating business requirements into a testable, documented CRM solution.

Appendix A: Full Object & Field Reference

This appendix lists every custom field created on each custom object in the org, for reference alongside the summarized tables in Phase 1 and Phase 2.

A.1 Vehicle_c

Field API Name	Data Type	Description
Make_c	Text(60)	Manufacturer name, e.g. 'Astra Motors'.
Model_c	Text(80)	Model name, e.g. 'Astra GT'.
Variant_c	Text(60)	Trim/variant, e.g. 'GT Sport Automatic'.
Branch_c	Lookup(Account)	Branch where this unit is stocked.
Price_c	Currency(16,2)	List price.
Available_Units_c	Number(6,0)	Current stock count used for eligibility checks.
Fuel_Type_c	Picklist	Petrol / Diesel / Electric / Hybrid.
Is_Demo_Unit_c	Checkbox	Flags a unit reserved specifically for test drives.

A.2 Dealer_c

Field API Name	Data Type	Description
Branch_c	Lookup(Account)	Branch the dealer is based at.
User_c	Lookup(User)	Related Salesforce User record for the dealer.
Active_c	Checkbox	Whether the dealer is currently taking new bookings.
Current_Load_c	Roll-Up Summary	Count of open Test_Drive_c records assigned to this dealer.
Specialization_c	Picklist (Multi-Select)	Vehicle categories the dealer specializes in (SUV, Sedan, EV, etc.).

A.3 Test_Drive_c

Field API Name	Data Type	Description
Contact_c	Lookup(Contact)	Customer requesting the test drive.
Vehicle_c	Lookup(Vehicle_c)	Vehicle being test-driven.
Dealer_c	Lookup(Dealer_c)	Dealer assigned to handle the booking.

Field API Name	Data Type	Description
Branch_c	Lookup(Account)	Branch the booking belongs to (defaulted from Vehicle_c).
Status_c	Picklist	New / Assigned / Confirmed / Completed / Cancelled / No-Show / Expired / Unassigned – Needs Review.
Preferred_Date_Time_c	DateTime	Customer's requested slot.
Confirmed_Date_c	DateTime	Final agreed slot once confirmed.
Reminder_Sent_c	Checkbox	Guards against duplicate reminder emails.
Outcome_c	Picklist	Converted / Not Converted / Pending, captured after completion.
Discount_Requested_c	Percent	Discount percentage requested, feeds the Approval Process.

A.4 Payment_c

Field API Name	Data Type	Description
Test_Drive_c	Lookup(Test_Drive_c)	The converted booking this payment settles.
Booking_Amount_c	Currency(16,2)	Amount collected.
Payment_Mode_c	Picklist	Cash / Card / UPI / Bank Transfer.
Reference_Number_c	Text(40)	Transaction reference; restricted via field-level security.
Payment_Date_c	Date	Date payment was recorded.

Appendix B: Glossary of Terms

Term	Definition
Org-Wide Default (OWD)	The baseline level of record access (Private, Public Read Only, Public Read/Write) before any sharing rules or role hierarchy are applied.
Bulkification	Writing Apex so that DML and SOQL operations work correctly and efficiently on many records at once, not just one.

Governor Limits	Salesforce's per-transaction limits on resources such as SOQL queries, DML statements, and CPU time, designed to keep the multi-tenant platform stable.
Change Set	A packaged set of metadata components moved between two connected Salesforce orgs (e.g., Sandbox to Production).
Term	Definition
Dynamic Forms	A Lightning feature allowing fields and sections on a record page to be shown/hidden conditionally, replacing the need for multiple static page layouts.
Schedulable Apex	An Apex class implementing the Schedulable interface so it can run automatically at a specified time using System.schedule().
Batch Apex	A framework for processing large numbers of records asynchronously in manageable chunks via the Database.Batchable interface.
Field-Level Security (FLS)	Profile or permission-set-level control over whether a field is visible/editable for a given user, independent of object or record-level access.
Matching Rule	Defines how Salesforce compares field values (exact or fuzzy) to detect potential duplicate records.
Lightning Path	A visual component showing a picklist field's stage progression (e.g., a Status c value) across the top of a record page.

Appendix C: Sample Test Data

The following representative sample rows illustrate the kind of seed data used across UAT and the Apex bulk test setup, including the sample customer referenced in the Phase 2 walkthrough.

Contact	Vehicle	Branch	Status
Elijah Mikaelson	Astra GT Sport Automatic	Anna Nagar	Completed
Priya Raman	Voyager LX Diesel	T. Nagar	Confirmed
Arjun Nair	Voyager EV	Anna Nagar	Assigned
Sneha Kapoor	Astra GT Sport Automatic	Velachery	New
Rahul Verma	Trekker 4x4	T. Nagar	Cancelled

Appendix D: Team Roles & Responsibilities

Role	Responsibility on This Project
Salesforce Developer (Author)	End-to-end design and build: data model, Flows, Apex, security, testing, and deployment.
Project Mentor / Evaluator	Reviewed requirement analysis and validated the final demo and documentation.
Branch Operations SME (stakeholder)	Provided business requirements and validated UAT scenarios against real dealership workflows.

Appendix E: Sample Email Template

The following is the merge-field template used by the Test Drive Reminder Flow, shown here in plain text form (the production version is a branded HTML email template).

```
Subject: Reminder - Your Test Drive Tomorrow at {!Branch_c}

Hi {!Contact_c.FirstName},

This is a reminder for your upcoming test drive:

    Vehicle: {!Vehicle_c.Make_c} {!Vehicle_c.Model_c}
    Date/Time: {!Preferred_Date_Time_c}
    Branch: {!Branch_c.Name}
    Dealer: {!Dealer_c.Name}

If you need to reschedule, please call your branch directly.

We look forward to seeing you,
WhatNext Vision Motors
```

Appendix F: Dashboard Component Reference

For completeness, the individual components on the Sales Performance dashboard referenced in Phase 3 are listed below with their chart type and source report.

Component	Chart Type	Source Report
Bookings by Status	Donut Chart	Bookings by Status and Branch
Bookings by Branch	Vertical Bar Chart	Bookings by Status and Branch
Top Dealers by Conversion	Horizontal Bar Chart	Dealer Conversion Matrix
Monthly Booking Trend	Line Chart	Monthly Booking Trend
Total Confirmed Bookings (This Month)	Metric/Gauge	Bookings by Status and Branch

Appendix G: Frequently Asked Questions

This appendix collects a handful of questions raised repeatedly during stakeholder demos, along with the answers given at the time, to save re-explaining them in future reviews.

Q: What happens if two dealers are tied on Current_Load_c?

The Dealer Assignment Flow's Get Records element applies a secondary sort on Dealer_c Name, so ties are broken alphabetically and the assignment remains deterministic rather than random.

Q: Can a customer book more than one test drive at a time?

Yes — Test_Drive_c has no uniqueness constraint against Contact_c, since a genuine use case is a customer comparing two different vehicles before deciding. Reporting is unaffected because each booking is tracked as its own record.

Q: What happens to a Payment_c record if the linked Test_Drive_c is deleted?

The lookup from Payment_c to Test_Drive_c is configured to prevent deletion of the parent Test_Drive_c while a related Payment_c exists, protecting financial records from accidental loss.

Q: Why was Batch Apex used instead of a simple Scheduled Flow for stale-booking cleanup?

At the data volumes expected once all branches are live, a Scheduled Flow acting on the full stalerecord set risked exceeding per-transaction limits; Batch Apex's chunked execution was chosen specifically to remain safe as volume grows, per the constraint noted in Phase 1.

Appendix H: Sample Discount Approval Scenario

To make the Approval Process in Phase 2 concrete, a walkthrough scenario was used during UAT: a Sales Executive at the T. Nagar branch enters a discount request of 15% for a customer purchasing the Voyager LX Diesel, exceeding the branch's standard 10% authority. On save, the Test_Drive_c record is locked and Status_c moves to 'Pending Approval', and the request is routed to the T. Nagar Branch Manager's approval queue. The Branch Manager reviews the request against the customer's negotiation history and either approves it — unlocking the record, honoring the 15% discount, and stamping Approved_By_c and Approval_Date_c — or rejects it, which reverts Status c to 'Confirmed' at the original price and notifies the Sales Executive by email so the conversation with the customer can continue without delay.

This scenario was captured as part of the manual test evidence in Phase 4 and confirmed that the approval routing, record locking, and notification all behave correctly together, not just as isolated configuration.

Appendix I: References

- Salesforce Help – Batch Apex Developer Guide.
- Salesforce Help – Flow Builder Documentation.
- Salesforce Trailhead – Data Modeling and Security modules.
- Salesforce Help – Duplicate Management (Matching and Duplicate Rules).
- Internal stakeholder interviews with Sales Executive, Branch Manager, and Regional Manager personas conducted during Phase 1.